

STAYBUDDY – COMPLETE IMPLEMENTATION PLAN

April 23, 2026 → May 10, 2026

Team: Eraj Zaman, Samiya Saleem, Zarnab

TABLE OF CONTENTS

1. Big Picture Goal
2. What's Already Done
3. What's Left to Build
4. Team Roles & Ownership
5. Day-by-Day Master Schedule
6. Eraj's Complete Technical Breakdown
7. Samiya's Complete Technical Breakdown
8. Zarnab's Complete Technical Breakdown
9. Integration Checkpoints
10. Database Schema Reference
11. API Contract Reference
12. The Demo Flow (Target End State)
13. Testing Checklist
14. Presentation Structure
15. Emergency Backup Plans
16. Daily Standup Template

1. BIG PICTURE GOAL

By May 10th you need to walk into your evaluation and demonstrate a CONNECTED, LIVING system – not three separate notebooks and mockups. The evaluators should see one fluid story:

A student opens StayBuddy → gets AI-personalized hostel recommendations → asks the chatbot a question → the chatbot responds intelligently → the student books a room → the warden sees it on the dashboard → the parent gets notified.

Every feature you build between now and May 10th should serve this narrative. Anything that doesn't fit this story is a lower priority.

The system must demonstrate:

- [1] Working AI recommendation engine (already done – now needs a face)
- [2] Working NLP chatbot (nearly done – needs completion + UI)
- [3] Real database with real data flowing through real APIs (Zarnab's job)
- [4] A frontend that a human can click through during a demo

```
46 [5] At least partial warden and parent views
47 [6] Measurable ML metrics you can show on a slide
48
49 This document gives every team member a complete, unambiguous list of what to
50 build, in what order, with enough technical detail to execute without confusion.
51
52
53 =====
54 2. WHAT'S ALREADY DONE (Prelim Deliverables)
55 =====
56
57 Eraj:
58   ✓ Synthetic dataset (50-100 hostels, 200+ student profiles)
59   ✓ Content-based filtering model (cosine similarity on hostel features)
60   ✓ Collaborative filtering model (matrix factorization / KNN)
61   ✓ Hybrid recommendation model with weighted combination
62   ✓ Evaluation metrics: Precision@K, MAP, RMSE, Coverage
63   ✓ Jupyter notebook documenting everything
64
65 Samiya:
66   ✓ Intent categories defined (hostel search, amenity inquiry, pricing,
67     booking process, location info, complaint/issue, general info)
68   ✓ Training data created (20-30 examples per intent)
69   ✓ Intent classification model trained (DistilBERT or similar)
70   ~ Entity extraction – partially done or in progress
71   ~ Response templates – partially done
72   × Chatbot UI – not built
73   × Full end-to-end chatbot working via API – not done
74
75 Zarnab:
76   ✓ PostgreSQL database schema designed
77   ✓ Database implemented with constraints and indexes
78   ✓ Seeding scripts with synthetic data
79   ✓ Core REST API endpoints (hostels GET, user profiles)
80   ✓ ML model integration endpoints (stubs/placeholders)
81   ✓ API documentation
82   ~ All endpoints returning real DB data (some may still be mock)
83
84 What this means: The "brain" exists. Now you need the "body."
85
86
87 =====
88 3. WHAT'S LEFT TO BUILD
89 =====
90
```

```
91 MUST HAVE for May 10 evaluation:
92   - Chatbot fully working end-to-end with UI
93   - Student frontend: search, recommendation results, hostel detail, booking
94   - Warden dashboard: complaints list, attendance view, student list
95   - Parent portal: child status, notifications, attendance
96   - All APIs returning real database data (no more mocks)
97   - Authentication: login/register working for all roles
98   - Integration: frontend calling real APIs calling real ML models
99   - Demo-ready: scripted flow that works reliably
100
101 GOOD TO HAVE if time permits:
102   - Hostel owner listing management
103   - Admin verification portal
104   - Payment gateway (even a fake one that shows confirmation)
105   - Push notifications
106   - Map view with hostel pins
107   - Student profile completion flow
108   - Review/rating submission
109
110 DO NOT BUILD before May 10 (post-evaluation):
111   - Biometric/RFID hardware integration
112   - Real SMS/call emergency alerts
113   - Kubernetes/Docker production deployment
114   - Real payment processing
115   - Geofencing hardware
116   - Voice search (unless Eraj is ahead of schedule)
117
118
119 =====
120 4. TEAM ROLES & OWNERSHIP
121 =====
122
123 ERAJ owns:
124   - Recommendation engine API (already trained, needs serving)
125   - Student mobile/web app (all screens)
126   - Voice search (if time permits after student app done)
127   - AI model testing and integration verification
128
129 SAMIYA owns:
130   - Chatbot completion (entity extraction, response templates, API endpoint)
131   - Chatbot UI (embedded chat widget)
132   - Parent portal (all screens)
133   - Complaint submission flow from student side
134   - UI/UX consistency across the app
135
```

```
136 ZARNAB owns:
137   - All backend APIs (real DB responses, no mocks)
138   - Authentication system (JWT login/register for all 4 roles)
139   - Warden dashboard (web-based)
140   - Admin portal (basic version)
141   - Database integrity and performance
142   - Deployment (even if just localhost or a simple cloud server)
143
144 SHARED:
145   - Daily integration testing (30 min together every evening)
146   - Demo script rehearsal (last 3 days)
147   - Presentation slides
148
149
150 =====
151 5. DAY-BY-DAY MASTER SCHEDULE
152 =====
153
154 PHASE 1: COMPLETION (April 23-27) – "Close all open loops"
155 -----
156
157 APRIL 23 (TODAY – Wednesday)
158   Eraj:
159     - Set up FastAPI/Flask endpoint that serves the trained recommendation model
160     - Endpoint: POST /api/recommendations with {student_id} → returns ranked hostels
161     - Test this endpoint with Postman – it must return real results from your model
162     - Set up your frontend project (React Native Web or React + Expo)
163     - Build the app skeleton: navigation structure, placeholder screens
164
165   Samiya:
166     - Complete entity extraction (budget amounts, location names, amenity names)
167     - Test entity extraction on 20 diverse inputs, fix failures
168     - Build response generation function for all 7 intents
169     - Write all response templates (at least 2 per intent for variety)
170     - Create POST /api/chatbot endpoint that accepts {message, conversation_history}
171       and returns {response, intent_detected, entities_extracted}
172
173   Zarnab:
174     - Audit every API endpoint – mark which ones return real DB data vs mock
175     - For each mock endpoint: replace with real PostgreSQL query TODAY
176     - Implement JWT authentication: POST /auth/login, POST /auth/register
177     - Auth must support 4 roles: student, parent, warden, admin
178     - Test login with Postman for each role
179
180 APRIL 24 (Thursday)
```

181 Eraj:

182 - Build Student App Screen 1: Login/Register (connect to Zarnab's auth API)

183 - Build Student App Screen 2: Home/Search screen

184 (search bar, filter chips: price range, distance, amenities)

185 - Build Student App Screen 3: Recommendation Results

186 (card list showing hostel name, price, distance, match score %)

187 - Connect Screen 3 to your recommendation API endpoint

188

189 Samiya:

190 - Build the Chat UI component

191 (message bubbles, input bar, send button, typing indicator)

192 - Connect Chat UI to your chatbot API endpoint

193 - Test the full chatbot loop: type message → API processes → response appears

194 - Start Parent Portal Screen 1: Login and dashboard home

195 - Dashboard home shows: child's name, current hostel, last attendance time

196

197 Zarnab:

198 - Build Warden Dashboard Screen 1: Login and home overview

199 (total students, open complaints count, today's attendance rate)

200 - Build Warden Dashboard Screen 2: Complaints list

201 (table showing complaint ID, student name, category, priority, status)

202 - Implement PATCH /api/complaints/{id} endpoint for wardens to update status

203 - Implement GET /api/complaints?warden_id= endpoint

204

205 APRIL 25 (Friday)

206 Eraj:

207 - Build Student App Screen 4: Hostel Detail Page

208 (photos placeholder, amenities list, price breakdown, ratings, reviews)

209 - Build Student App Screen 5: Booking Flow Step 1

210 (room type selection, date picker, bed availability)

211 - Connect Hostel Detail to GET /api/hostels/{id} endpoint

212

213 Samiya:

214 - Integrate chatbot into the Student App as a floating button/chat screen

215 - Parent Portal Screen 2: Child's attendance history

216 (calendar view or list view showing check-in/check-out times)

217 - Parent Portal Screen 3: Notifications center

218 (list of alerts: attendance logged, complaint update, emergency)

219 - Test parent portal with real data from Zarnab's API

220

221 Zarnab:

222 - Build Warden Dashboard Screen 3: Student roster

223 (list of all students at the hostel with their room number and status)

224 - Build Warden Dashboard Screen 4: Attendance marking

225 (manually mark a student present/absent for demo purposes)

- Implement POST /api/attendance endpoint
- Implement GET /api/students?hostel_id= endpoint
- Ensure all hostel data endpoints include amenities, room types, pricing

APRIL 26 (Saturday – INTEGRATION DAY 1)

ALL THREE TOGETHER for at least 4-6 hours:

- Eraj calls Zarnab's APIs from the student app – fix any mismatches
- Samiya calls Zarnab's APIs from parent portal – fix any mismatches
- Chatbot endpoint integrated into main API gateway
- Recommendation endpoint integrated into main API gateway
- Agree on a SHARED BASE URL for all API calls
- Test the following mini-flow end to end:
Student logs in → sees recommendations → clicks a hostel → views detail
- Document every bug found, assign fixes

APRIL 27 (Sunday – INTEGRATION DAY 2)

Eraj:

- Fix all bugs from Saturday's integration session
- Build Student App Screen 6: Booking confirmation screen
(booking summary, total price, confirm button → success screen)
- Implement POST /api/bookings endpoint call from frontend
- Polish search screen filters (make them actually filter results)

Samiya:

- Fix all bugs from Saturday's integration session
- Parent Portal Screen 4: Warden contact
(warden name, phone, in-app message button – can be fake send for now)
- Chatbot: add complaint filing intent that creates a real complaint in the DB
(this means chatbot calls POST /api/complaints when complaint intent detected)
- Ensure chatbot handles unknown/out-of-scope questions gracefully

Zarnab:

- Fix all bugs from Saturday's integration session
- Implement GET /api/notifications?user_id= endpoint
(returns list of notifications for parent portal)
- Implement POST /api/bookings and verify it creates a booking in the DB
- Verify referential integrity: booking links to real student and real hostel
- Set up CORS properly so all frontends can call the backend

PHASE 2: FEATURE COMPLETION (April 28 - May 3) – "Build everything remaining"

APRIL 28 (Monday)

271 Eraj:

- 272 - Student App: Add hostel map view screen
- 273 (use Google Maps / Leaflet.js with hostel pins)
- 274 - Student App: Add student profile screen
- 275 (name, university, budget preference, amenity preferences, edit button)
- 276 - Connect profile edit to PUT /api/students/{id} endpoint
- 277

278 Samiya:

- 279 - Complaint submission flow from student app:
- 280 Screen: "File a Complaint" with category dropdown, description text box,
- 281 severity selector (low/medium/high), submit button
- 282 - This should call POST /api/complaints and confirm submission
- 283 - Add complaint tracking screen: list of student's own complaints with status
- 284 - Parent Portal: add fee payment tracking screen
- 285 (shows outstanding amount, due date – data can be static for demo)
- 286

287 Zarnab:

- 288 - Admin Portal Screen 1: Login and overview
- 289 (total hostels, total students, open disputes, pending verifications)
- 290 - Admin Portal Screen 2: Hostel verification list
- 291 (table of hostels with status: pending, approved, rejected)
- 292 - Implement PATCH /api/hostels/{id}/verify endpoint
- 293 - Add admin role to JWT middleware
- 294

295 APRIL 29 (Tuesday)

296 Eraj:

- 297 - Student App: My Bookings screen
- 298 (list of current and past bookings with status)
- 299 - Student App: Reviews section on hostel detail
- 300 (show existing reviews, add review button with rating stars + text)
- 301 - Implement POST /api/reviews endpoint call from frontend
- 302

303 Samiya:

- 304 - Warden Dashboard: Complaint detail view
- 305 (full complaint info, student details, AI-suggested resolution shown,
- 306 status dropdown for warden to update, notes field)
- 307 - NOTE: The AI-suggested resolution should come from your complaint
- 308 categorization model – feed the complaint text to your NLP model
- 309 and return a suggested action
- 310 - Connect this to Zarnab's PATCH /api/complaints/{id} endpoint
- 311

312 Zarnab:

- 313 - Implement GET /api/complaints/{id} with full detail
- 314 - Add complaint_suggestion field to complaint model
- 315 (populated by Samiya's NLP model when complaint is created)

- Implement GET /api/analytics/warden?hostel_id= returning:
total complaints this month, resolution rate, avg resolution time,
attendance rate this week
- Warden Dashboard: Analytics screen using above endpoint

APRIL 30 (Wednesday)

Eraj:

- Add "Why was this recommended?" tooltip/explanation on recommendation cards
(show top 3 matching factors: e.g., "within your budget," "has WiFi,"
"popular with CS students")
- Add loading skeletons and empty states to all screens
- Fix any UI inconsistencies: colors, fonts, spacing

Samiya:

- Chatbot: Add context awareness across multiple messages
(store last 3 messages in conversation history, send to API)
- Chatbot: Add quick reply buttons for common follow-ups
(e.g., after pricing response, show "Book Now" and "See Similar" buttons)
- Complete chatbot confusion matrix and F1 metrics – generate final report
- Embed final chatbot metrics into a simple display in the app (for demo)

Zarnab:

- Emergency SOS: Implement POST /api/emergency endpoint
(creates emergency record with student_id, location, timestamp, type)
- Add SOS button to student app (big red button on home screen)
- Warden gets notified (new record in their notification feed)
- Parent gets notified (new record in their notification feed)
- This can be simulated without real SMS – just DB records shown in UI

MAY 1 (Thursday)

Eraj:

- Student App: Favorites/Saved Hostels screen
(hostels the student has heart-favorited)
- Implement POST /api/favorites and GET /api/favorites?student_id=
- Add heart icon to hostel cards that toggles favorite status
- Begin testing the full student journey: register → search → recommend →
view hostel → chat → book → review

Samiya:

- Parent Portal: Live location screen
(for demo, show a static Google Maps embed with the hostel location
marked – label it "Last known location: [hostel name]")
- Parent Portal: Emergency alert screen
(shows red banner when an SOS is active, with student name and location)
- Polish all parent portal screens for demo readiness

361
362 Zarnab:
363 - Implement GET /api/hostel-availability?hostel_id=&room_type=
364 (returns available beds count)
365 - Implement GET /api/dashboard/student?student_id=
366 (returns student's booking, attendance summary, open complaints)
367 - Full end-to-end test of all Zarnab's APIs with a test script
368 - Fix all failing endpoints
369
370 MAY 2 (Friday)
371 Eraj:
372 - Voice search (if time permits – otherwise skip and note as future work)
373 - If building: use Web Speech API in browser (free, no API key needed)
374 Simply converts speech to text, then feeds text into the search bar
375 - Student app performance pass: remove console logs, fix slow renders
376 - Make sure the app works on mobile browser (responsive design check)
377
378 Samiya:
379 - Full chatbot stress test: 50 diverse queries covering all intents
380 - Fix any intent misclassification found
381 - Warden Dashboard: Add AI-suggestions banner at top of complaints page
382 (shows pattern detection – e.g., "5 WiFi complaints this week – possible
383 infrastructure issue")
384 - This can be rule-based for now: if 3+ complaints of same category in 7 days,
385 show the banner
386
387 Zarnab:
388 - Set up the application for demo deployment
389 - Options: localhost (all on same network), Render.com free tier,
390 Railway.app, or ngrok tunnel for demo day
391 - Make sure the backend is accessible from all team members' devices
392 - Database backup: export a clean seed of the demo database
393 - Write a one-command startup script: "npm start" or "python run.py"
394 that starts backend, serves frontend, ready to demo
395
396 MAY 3 (Saturday – INTEGRATION DAY 3 – Full System Test)
397 ALL THREE TOGETHER – Full day:
398 - Run through the complete demo flow (see Section 12)
399 - Every person on a different device (phone + laptop + tablet or 3 laptops)
400 - Student app on one device, warden dashboard on another, parent on third
401 - Execute the full story from start to finish
402 - Document every bug, every broken link, every wrong data
403 - Divide bugs by owner, fix urgently
404 - By end of day: demo flow must work without crashes
405 - Record a backup video of the demo working (in case of demo day technical issues)

406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450

PHASE 3: POLISH & DEMO PREP (May 4-7) – "Make it shine"

MAY 4 (Sunday)

Eraj:

- Fix all bugs from Saturday's full system test
- Final recommendation model check: run evaluation metrics one more time
- Generate clean output of metrics for presentation slide
- Add "AI Insights" section to student home screen
(small card showing: "Based on your preferences: X% match rate,
Top recommendation: [hostel name]")

Samiya:

- Fix all chatbot bugs from Saturday
- Generate final confusion matrix, F1 scores, accuracy report
- Make these metrics visually presentable (can be a screenshot from notebook)
- Final parent portal polish: make it look finished

Zarnab:

- Fix all backend bugs from Saturday
- Generate API performance report:
Run 100 requests to each endpoint, calculate average response time
(use a simple Python script with requests library)
- Final database consistency check: no orphaned records
- Prepare the demo database with clean, realistic looking data
(give hostels real-sounding Islamabad names, students realistic names)

MAY 5 (Monday)

ALL THREE:

- Start building the presentation (see Section 14 for structure)
- Build the architecture diagram for the slides (show all 3 layers connecting)
- Each person prepares their own 5-minute technical explanation of their module
- Practice explaining your work without jargon to a non-technical audience

MAY 6 (Tuesday)

ALL THREE:

- Complete presentation slides
- FEATURE FREEZE: No new features after today
- Only bug fixes allowed from here on
- Second full run-through of demo flow
- Time it: target 8-10 minutes for full demo, 5 minutes for presentation

```
451 MAY 7 (Wednesday)
452     ALL THREE:
453     - Full dress rehearsal: one person presents, others watch and take notes
454     - Fix any last bugs discovered
455     - Prepare backup plans (see Section 15)
456     - Upload final code to GitHub with clean commit history
457     - Make sure README has setup instructions
458
459 -----
460
461 PHASE 4: FINAL PREPARATION (May 8-10)
462 -----
463
464 MAY 8 (Thursday)
465     - Final bug fixes only
466     - Rehearse demo and presentation one more time
467     - Prepare Q&A answers (see Section 14)
468     - Print or have ready: architecture diagram, metrics screenshots
469
470 MAY 9 (Friday)
471     - Rest day – seriously, sleep
472     - Light review of your own component only
473     - Make sure demo devices are charged and working
474     - Have the backup video ready on your phone
475
476 MAY 10 (Saturday – EVALUATION DAY)
477     - Arrive early, set up devices
478     - Do one quick smoke test before evaluators arrive
479     - Execute the demo exactly as rehearsed
480     - Present metrics confidently
481
482
483 =====
484 6. ERAJ'S COMPLETE TECHNICAL BREAKDOWN
485 =====
486
487 YOUR MAIN JOB: Serve the recommendation model via API and build the student app.
488
489 ----- A. RECOMMENDATION MODEL API ENDPOINT -----
490
491 Framework: FastAPI (recommended) or Flask
492
493 File structure:
494     ml_service/
495     app.py
```

```
496     model_loader.py      (loads your trained sklearn/numpy model on startup)
497     recommender.py       (the actual prediction logic)
498     requirements.txt
499
500 model_loader.py:
501     - Load your trained content-based model (pickle or joblib file)
502     - Load your trained collaborative filtering model
503     - Load your hostel feature matrix
504     - Load your student-hostel interaction matrix
505     - Keep all loaded in memory – don't reload per request
506
507 Endpoint 1: POST /api/recommendations
508     Input: { "student_id": 123, "top_k": 10 }
509     Output: {
510         "recommendations": [
511             {
512                 "hostel_id": 45,
513                 "hostel_name": "Green Valley",
514                 "match_score": 0.87,
515                 "content_score": 0.82,
516                 "collaborative_score": 0.91,
517                 "top_matching_factors": ["within budget", "has WiFi", "near campus"],
518                 "price_per_month": 12000,
519                 "distance_km": 0.8,
520                 "rating": 4.2
521             }
522         ]
523     }
524
525 The "top_matching_factors" field is IMPORTANT for demo.
526 Generate it by checking which hostel features best match student preferences:
527     - If hostel price <= student budget → add "within budget"
528     - If hostel has wifi and student wants wifi → add "has WiFi"
529     - If distance < student's commute tolerance → add "near campus"
530     - If hostel has study room and student prefers study env → add "study-friendly"
531     Maximum 3 factors per hostel.
532
533 Endpoint 2: GET /api/recommendations/profile/{student_id}
534     Returns the student's preference profile
535     (so the frontend can show "Your preferences: budget 15000, prefers WiFi, quiet")
536
537 Endpoint 3: PUT /api/recommendations/profile/{student_id}
538     Updates student preferences
539     (so students can update their profile and get new recommendations)
540
```

541 Run this as a separate microservice on a different port (e.g., port 8001)

542 Zarnab's main API can call it internally or you can expose it directly.

543

544 ----- B. STUDENT APP SCREENS -----

545

546 Use React (web) or React Native (mobile) – whichever you're more comfortable with.

547 For demo purposes, a responsive web app works perfectly fine.

548

549 Stack recommendation: React + Tailwind CSS + Axios (for API calls)

550 This is fast to build and looks great.

551

552 SCREEN 1: Welcome / Splash

- 553 - StayBuddy logo (use the green color from your proposal: #2D6A4F or similar)
- 554 - "Your Intelligent Hostel Companion"
- 555 - Login button, Register button

556

557 SCREEN 2: Login

- 558 - Email input, password input, Login button
- 559 - Role selector (student, parent, warden, admin) – dropdown
- 560 - Calls: POST /auth/login → stores JWT token in state
- 561 - On success: routes to role-specific home

562

563 SCREEN 3: Student Registration

- 564 - Name, email, password, university, phone
- 565 - Budget range slider (5000 to 30000 PKR)
- 566 - Amenity preferences checkboxes: WiFi, AC, Meals, Study Room, Gym, Laundry
- 567 - Commute tolerance: < 0.5km, < 1km, < 2km, any
- 568 - Social vs Study preference slider
- 569 - Calls: POST /auth/register with all this data

570

571 SCREEN 4: Student Home / Dashboard

- 572 - Greeting: "Welcome back, [name]"
- 573 - Search bar (text input + search icon + mic icon for voice if implemented)
- 574 - Quick filters: price range, distance, amenities (horizontal scroll chips)
- 575 - Section: "Recommended for You" – horizontal scroll of hostel cards
- 576 - Section: "Recently Viewed" – horizontal scroll
- 577 - SOS button (red, bottom of screen or floating)

578

579 SCREEN 5: Search Results / All Recommendations

- 580 - List of hostel cards, each showing:
 - 581 - Hostel name
 - 582 - Price per month
 - 583 - Distance from campus
 - 584 - Star rating (X.X / 5)
 - 585 - Match score badge: "87% Match" in green

- Top matching factor chips: "WiFi" "Within Budget" "Near Campus"
- Heart icon (toggle favorite)
- Sort by: Match Score, Price Low-High, Price High-Low, Rating, Distance
- Filter panel (slide from right): price range, amenities, distance, room type

SCREEN 6: Hostel Detail

- Photo placeholder (grey box with hostel name, or use a placeholder image URL)
- Hostel name, address, overall rating
- Tab bar: Overview | Amenities | Rooms | Reviews | Location

Overview tab:

- Description paragraph
- Warden name and contact
- Distance from campus
- Match score with breakdown bar chart

Amenities tab:

- Grid of amenity icons: WiFi ✓, AC ✓, Meals ✓, Gym ✗, etc.

Rooms tab:

- List of room types: Single, Double, Triple, Dormitory
- For each: price/month, availability count, Book Now button

Reviews tab:

- Average rating breakdown (cleanliness, facilities, management, location)
- List of student reviews with star rating and text
- "Add Review" button (only if student has a booking here)

Location tab:

- Google Maps embed showing hostel pin and campus pin
- Distance and estimated walking/driving time

SCREEN 7: Booking Flow

Step 1 – Select Room:

- Room type dropdown
- Check-in date (calendar picker – just a date input)
- Duration: 1 month, 3 months, 6 months, 1 year
- Price calculation shown dynamically

Step 2 – Confirm Details:

- Summary: hostel name, room type, duration, total price
- Special requirements text box
- Confirm Booking button
- Calls: POST /api/bookings

```
631 Step 3 – Success Screen:
632   - Green checkmark animation
633   - Booking ID
634   - "Your booking is confirmed at [hostel name]"
635   - Back to Home button
636
637 SCREEN 8: My Bookings
638   - Tabs: Active | Past | Pending
639   - Each booking card: hostel name, room type, price, dates, status badge
640   - Cancel button on pending bookings
641
642 SCREEN 9: Chat (AI Assistant)
643   - Full screen chat interface
644   - Message bubbles (yours on right in green, bot on left in grey)
645   - Input bar at bottom with text field and send button
646   - Quick reply chips after bot messages
647   - Typing indicator (three dots) while waiting for API response
648   - First message from bot: "Hi! I'm your StayBuddy assistant.
649     Ask me anything about hostels, bookings, or accommodation."
650
651 SCREEN 10: My Complaints
652   - List of complaints filed by the student
653   - Status badge: Pending (orange), In Progress (blue), Resolved (green)
654   - "File New Complaint" button
655
656 SCREEN 11: File Complaint
657   - Category: Maintenance | Food | Cleanliness | Security | Internet | Other
658   - Severity: Low | Medium | High
659   - Description: multi-line text input
660   - Photo attachment option (for demo, just a file input)
661   - Submit button → calls POST /api/complaints
662
663 SCREEN 12: Profile / Settings
664   - Profile photo placeholder
665   - Name, university, contact
666   - Update Preferences button (goes to preference update screen)
667   - Current hostel info (if booked)
668   - Logout button
669
670 ----- C. COMPONENT ARCHITECTURE -----
671
672 src/
673   components/
674     HostelCard.jsx           (reusable card for recommendation lists)
675     ChatBubble.jsx           (single message in chat)
```

```

676     AmenityIcon.jsx          (icon + label for amenities)
677     StarRating.jsx          (star display component)
678     BookingStatusBadge.jsx  (colored status indicator)
679     LoadingSkeleton.jsx     (placeholder while data loads)
680 screens/
681     Auth/
682         LoginScreen.jsx
683         RegisterScreen.jsx
684     Student/
685         HomeScreen.jsx
686         SearchResultsScreen.jsx
687         HostelDetailScreen.jsx
688         BookingScreen.jsx
689         ChatScreen.jsx
690         ComplaintsScreen.jsx
691         ProfileScreen.jsx
692 services/
693     api.js                   (base axios instance with JWT header)
694     authService.js          (login, register, logout)
695     hostelService.js        (search, get detail, favorites)
696     recommendationService.js (get recommendations)
697     chatService.js          (send message, get response)
698     bookingService.js       (create, get, cancel)
699     complaintService.js     (create, get list)
700 context/
701     AuthContext.jsx         (global auth state: user, token, role)
702 navigation/
703     AppNavigator.jsx        (route guard based on role)
704
705 ----- D. API SERVICE PATTERN -----
706
707 api.js:
708     const BASE_URL = 'http://localhost:8000/api'; // change for deployment
709
710     const apiClient = axios.create({ baseURL: BASE_URL });
711
712     // Attach JWT token to every request:
713     apiClient.interceptors.request.use(config => {
714         const token = localStorage.getItem('token'); // or from context
715         if (token) config.headers.Authorization = `Bearer ${token}`;
716         return config;
717     });
718
719 hostelService.js:
720     export const getRecommendations = (studentId) =>

```



```
721     apiClient.post('/recommendations', { student_id: studentId });
722
723     export const getHostelDetail = (hostelId) =>
724         apiClient.get(`/hostels/${hostelId}`);
725
726     export const searchHostels = (filters) =>
727         apiClient.get('/hostels', { params: filters });
728
729     ----- E. MATCH SCORE DISPLAY -----
730
731     This is important for showing your AI works. On every hostel card show:
732         - A percentage: "87% Match"
733         - Color coded: 80%+ green, 60-79% yellow, below 60% grey
734         - On hover/click: show the top 3 matching factors as small chips
735
736     This makes your ML work VISIBLE to evaluators.
737
738
739     =====
740     7. SAMIYA'S COMPLETE TECHNICAL BREAKDOWN
741     =====
742
743     YOUR MAIN JOB: Complete the chatbot and build the parent portal.
744
745     ----- A. CHATBOT COMPLETION -----
746
747     ENTITY EXTRACTION (finish this first):
748         You need to extract these entities from student messages:
749
750         Budget amount:
751             - Patterns: "15000 rupees", "15k", "under 20,000", "below 15000 PKR"
752             - Approach: regex for numbers + currency keywords, or spaCy NER
753             - Extract: numeric value, operator (under/above/around)
754
755         Location:
756             - Patterns: "near FAST", "near the library", "F-7", "G-11", "Blue Area"
757             - Approach: maintain a list of known locations/landmarks near your campus
758             Match against this list, also handle "near [any word]"
759             - Extract: location string
760
761         Hostel name:
762             - Patterns: "Green Valley Hostel", "City View", "Sunrise"
763             - Approach: maintain list of hostel names in your DB, fuzzy match against it
764             - Extract: hostel_id + hostel_name
765
```

766 Amenity:
767 - Patterns: "WiFi", "gym", "meals", "study room", "AC", "laundry"
768 - Approach: simple keyword lookup dictionary
769 - Extract: list of amenity names
770
771 Room type:
772 - Patterns: "single", "double", "shared", "dormitory"
773 - Extract: room type string
774
775 Time reference:
776 - Patterns: "next month", "from January", "starting in May"
777 - Extract: date or relative time reference
778
779 RESPONSE TEMPLATES (write these out fully):
780
781 Intent: hostel_search
782 Template A (with budget + location):
783 "I found {count} hostels near {location} within {budget} PKR/month.
784 Here are the top matches: {hostel_list}"
785
786 Template B (with only budget):
787 "Here are hostels within your budget of {budget} PKR:
788 {hostel_list}"
789
790 Template C (with only location):
791 "Here are hostels near {location}: {hostel_list}"
792
793 Template D (no filters):
794 "Here are some popular hostels that students are choosing:
795 {hostel_list}"
796
797 Intent: amenity_inquiry (user asking if a hostel has something)
798 Template A (hostel has amenity):
799 "Yes! {hostel_name} does offer {amenity}.
800 It's included in the monthly rent."
801
802 Template B (hostel doesn't have amenity):
803 "Unfortunately, {hostel_name} doesn't have {amenity}.
804 Would you like me to find hostels that do?"
805
806 Template C (checking for user – no hostel specified):
807 "Here are hostels with {amenity}: {hostel_list}"
808
809 Intent: pricing_information
810 Template A (specific hostel):

811 "At {hostel_name}, room prices are:
812 Single: {single_price}/month
813 Double: {double_price}/month
814 Would you like to book, or see similar options?"
815
816 Template B (general cheapest):
817 "The most affordable option currently is {hostel_name}
818 starting at {price}/month."
819
820 Intent: booking_process
821 Template A (how to book):
822 "To book a hostel on StayBuddy:
823 1. Browse to the hostel's detail page
824 2. Select your room type and duration
825 3. Confirm your details
826 4. Your booking is confirmed instantly!
827 Would you like me to take you to a specific hostel?"
828
829 Template B (cancel booking):
830 "To cancel a booking, go to My Bookings, select the booking,
831 and tap Cancel. Refunds are processed within 3-5 business days."
832
833 Intent: location_information
834 Template A (distance from campus):
835 "{hostel_name} is approximately {distance} km from campus.
836 That's about {minutes} minutes walking / {drive_min} minutes by transport."
837
838 Template B (which hostels are walking distance):
839 "These hostels are within walking distance (< 1km) from campus:
840 {hostel_list}"
841
842 Intent: complaint_or_issue
843 Template A (filing a complaint):
844 "I'm sorry to hear that. I've logged your complaint about {issue_type}.
845 Your complaint ID is #{complaint_id}.
846 The warden will be notified and you'll receive updates."
847
848 Template B (guidance):
849 "For issues with {category}, you can:
850 1. File a complaint through the app (I can help with that)
851 2. Contact your warden directly
852 Would you like me to file a complaint now?"
853
854 Intent: general_information
855 Template A (visiting hours):

"Hostel visiting hours are typically 9 AM to 9 PM.
However, each hostel may have different rules.
I'd recommend checking with your warden for exact timings."

Template B (documents needed):

"To register at a hostel, you typically need:

- CNIC / B-Form copy
- University enrollment letter
- 2 passport photos
- Emergency contact information"

Intent: unknown (fallback):

"I'm not sure I understood that. I can help you with:
finding hostels, checking amenities, pricing, bookings,
location info, or filing complaints. What would you like to know?"

CONTEXT MANAGEMENT:

Your API should accept conversation history:

POST /api/chatbot

Input:

```
{  
  "message": "How far is it from campus?",  
  "conversation_history": [  
    {"role": "user", "message": "Tell me about Green Valley Hostel"},  
    {"role": "bot", "message": "Green Valley is located at..."}  
  ],  
  "student_id": 123 (optional, for personalized responses)  
}
```

Output:

```
{  
  "response": "Green Valley is approximately 0.8km from campus...",  
  "intent": "location_information",  
  "entities": {"hostel_name": "Green Valley", "location_ref": "campus"},  
  "quick_replies": ["View Hostel", "Book Now", "See Similar"]  
}
```

In your chatbot logic:

- If user message refers to "it" or "this hostel" – check conversation history for the last mentioned hostel name
- If user says "book it" – check for last mentioned hostel and trigger booking intent

COMPLAINT FILING VIA CHATBOT:

When complaint intent is detected:

1. If enough info in message: automatically call POST /api/complaints
with: { student_id, category (from entity), description (user message),
severity: "medium" (default) }
2. Return complaint ID in response
3. If not enough info: ask clarifying question
"What category is your complaint? (Maintenance/Food/Cleanliness/Security)"

PATTERN DETECTION (for warden dashboard banner):

```
Query DB: SELECT category, COUNT(*) FROM complaints
WHERE hostel_id = X AND created_at > NOW() - INTERVAL '7 days'
GROUP BY category HAVING COUNT(*) >= 3
```

If any category has 3+ complaints this week → trigger warning banner

----- B. PARENT PORTAL SCREENS -----

SCREEN 1: Parent Login

- Email, password, Login button
- Calls POST /auth/login with role=parent
- Shows friendly message: "Manage your child's hostel stay"

SCREEN 2: Parent Home Dashboard

Top section – Child Status Card:

- Child's photo placeholder
- Child's name
- Current hostel name and address
- Last check-in time ("Last seen: Today, 9:32 PM at Green Valley Hostel")
- Status indicator: green dot (in hostel) / orange dot (last seen X hours ago)

Middle section – Quick Stats:

- Attendance this week: 6/7 days present
- Open complaints: 1
- Next fee due: May 15 (PKR 12,000)

Bottom section – Recent Notifications:

- List of last 5 notifications with timestamps
- "View All" link

SCREEN 3: Attendance History

- Month selector at top
- Calendar view: green day (present), red day (absent), grey (no data)
- Below calendar: list view of each day with exact check-in/check-out times
- Export as PDF button (can be fake for demo – just show a button)

SCREEN 4: Notifications Center

```

946 - List of all notifications grouped by date
947 - Notification types with icons:
948   ✅ Attendance logged – "Ahmed checked in at 10:05 PM"
949   ⚠️ Complaint update – "Complaint #45 has been resolved"
950   🚨 Emergency – "SOS alert triggered by Ahmed at [time]"
951   💰 Fee reminder – "Monthly fee of PKR 12,000 due in 5 days"
952 - Tap notification → goes to relevant detail screen
953 - Unread notifications in bold
954
955 SCREEN 5: Live Location
956 - Google Maps embed
957 - Hostel location marked with a green pin labeled "Ahmed's Hostel"
958 - Campus location marked with a blue pin labeled "FAST University"
959 - Distance shown: "1.2 km from campus"
960 - Last updated timestamp: "Location last updated: 10:05 PM today"
961 - NOTE: For demo, this is the hostel's stored GPS coordinates,
962   not real-time phone GPS. Label it "Current Hostel Location"
963
964 SCREEN 6: Contact Warden
965 - Warden's photo placeholder
966 - Warden name, phone number
967 - "Call Warden" button (tel: link)
968 - "Send Message" button (can be a composed message screen that confirms sent)
969
970 SCREEN 7: Emergency Alert Screen
971 - Only shown when SOS is active
972 - Red banner: "⚠️ EMERGENCY ALERT"
973 - Student name, time of alert, type
974 - Map with location
975 - Contact options: Call Student, Call Warden, Call Emergency Services
976 - "Mark as Resolved" button (for when it's a false alarm)
977
978 SCREEN 8: Fee Tracking
979 - Table: Month | Amount | Due Date | Status (Paid/Pending/Overdue)
980 - Status badges in color: green (paid), orange (pending), red (overdue)
981 - "Pay Online" button on pending rows (for demo, just shows a success modal)
982
983 ----- C. WARDEN COMPLAINT DETAIL WITH AI SUGGESTION -----
984
985 When a complaint is submitted, your NLP model should process the description
986 text and return a suggested action. Here's how to implement this:
987
988 complaint_analyzer.py:
989     def get_ai_suggestion(complaint_text, category):
990         suggestions = {

```

```

991     "Maintenance": [
992         "Schedule a maintenance visit within 24 hours",
993         "Contact the building contractor for urgent repair",
994         "Temporarily relocate student if the issue is severe"
995     ],
996     "Food": [
997         "Review the meal with the kitchen staff",
998         "Offer alternative meal option temporarily",
999         "Investigate food quality with the supplier"
1000     ],
1001     "Internet/WiFi": [
1002         "Reset the router on floor {floor}",
1003         "Contact the ISP for a service check",
1004         "Provide data allowance as temporary solution"
1005     ],
1006     "Cleanliness": [
1007         "Schedule emergency cleaning for the reported area",
1008         "Review cleaning staff schedule and assignments",
1009         "Add the area to daily cleaning checklist"
1010     ],
1011     "Security": [
1012         "Conduct immediate security assessment",
1013         "Review CCTV footage from reported time",
1014         "Increase security patrols in reported area"
1015     ]
1016 }
1017 return suggestions.get(category, ["Review complaint and contact student"])[0]
1018

```

1019 This function gets called when a complaint is created:

- 1020 - Store suggestion in complaints table (field: ai_suggestion)
- 1021 - Return it in GET /api/complaints/{id}

1022
1023 Show it on the Warden Complaint Detail screen as:

```

1024     "💡 AI Suggested Action: Schedule a maintenance visit within 24 hours"
1025     In a blue highlighted box
1026

```

1027 This makes your AI involvement VISIBLE in a second place beyond recommendations.

1028
1029
1030 =====

1031 8. ZARNAB'S COMPLETE TECHNICAL BREAKDOWN

1032 =====

1033
1034 YOUR MAIN JOB: Solid backend, all real data, warden dashboard, auth system.

1035

1081			room_type, rating_min
1082	GET	/api/hostels/{id}	full detail with amenities and rooms
1083	POST	/api/hostels	admin only: create hostel
1084	PATCH	/api/hostels/{id}	warden/admin: update
1085	PATCH	/api/hostels/{id}/verify	admin only: approve/reject
1086	GET	/api/hostels/{id}/availability	returns room type availability counts
1087			
1088	RECOMMENDATIONS:		
1089	POST	/api/recommendations	calls ML service, returns ranked list
1090	GET	/api/recommendations/profile/{student_id}	
1091	PUT	/api/recommendations/profile/{student_id}	
1092			
1093	BOOKINGS:		
1094	POST	/api/bookings	creates booking
1095	GET	/api/bookings?student_id=	student's bookings
1096	GET	/api/bookings?hostel_id=	warden's hostel bookings
1097	GET	/api/bookings/{id}	
1098	PATCH	/api/bookings/{id}/status	cancel/confirm
1099			
1100	COMPLAINTS:		
1101	POST	/api/complaints	creates complaint, calls NLP for suggestion
1102	GET	/api/complaints?student_id=	student's own complaints
1103	GET	/api/complaints?hostel_id=	warden's hostel complaints
1104	GET	/api/complaints/{id}	
1105	PATCH	/api/complaints/{id}	warden updates status, adds notes
1106	GET	/api/complaints/patterns?hostel_id=	returns category counts for alerts
1107			
1108	ATTENDANCE:		
1109	POST	/api/attendance	log check-in or check-out
1110	GET	/api/attendance?student_id=&month=	student's attendance history
1111	GET	/api/attendance/summary?hostel_id=&date=	daily summary for warden
1112			
1113	REVIEWS:		
1114	POST	/api/reviews	student submits review
1115	GET	/api/reviews?hostel_id=	get hostel's reviews
1116			
1117	FAVORITES:		
1118	POST	/api/favorites	add hostel to favorites
1119	DELETE	/api/favorites/{hostel_id}	remove from favorites
1120	GET	/api/favorites?student_id=	get student's favorites
1121			
1122	NOTIFICATIONS:		
1123	GET	/api/notifications?user_id=	get user's notifications
1124	PATCH	/api/notifications/{id}/read	mark as read
1125	POST	/api/notifications	internal: create notification (not public)

```
1126
1127 EMERGENCY:
1128     POST    /api/emergency                trigger SOS
1129     GET     /api/emergency?hostel_id=      active emergencies for warden
1130     PATCH   /api/emergency/{id}/resolve
1131
1132 ANALYTICS:
1133     GET     /api/analytics/warden?hostel_id=
1134     Returns: {
1135         total_students, total_complaints_this_month,
1136         complaint_resolution_rate, avg_resolution_hours,
1137         attendance_rate_this_week, common_complaint_categories
1138     }
1139
1140 CHATBOT:
1141     POST    /api/chatbot                  routes to Samiya's NLP service
1142
1143 ----- C. DATABASE SCHEMA (COMPLETE) -----
1144
1145 See Section 10 for full schema. Implement these exact tables.
1146
1147 Key things to get right:
1148     1. Indexes on foreign keys (hostel_id, student_id) – critical for query speed
1149     2. created_at timestamp on every table – essential for analytics queries
1150     3. status enums enforced at DB level not just app level
1151     4. Cascade deletes where appropriate (e.g., delete favorites when hostel deleted)
1152
1153 ----- D. WARDEN DASHBOARD -----
1154
1155 Tech: React.js web app (separate from student mobile app, but same API)
1156 You can share components between them if you structure it right.
1157
1158 SCREEN 1: Warden Login
1159     - Simple email/password form
1160     - Role automatically detected from JWT token
1161
1162 SCREEN 2: Warden Home Dashboard
1163     Stats cards row (4 cards):
1164     - Total Students: [number]
1165     - Open Complaints: [number] (clickable → goes to complaints list)
1166     - Today's Attendance: [X/Y] (clickable → goes to attendance)
1167     - Alerts: [number in red if any emergencies active]
1168
1169     Below stats: Recent Activity Feed
1170     - Chronological list: "Ahmed filed a complaint • 2 hours ago"
```

1171 - "Fatima checked in • 5:30 PM today"

1172 - "Complaint #45 was resolved • Yesterday"

1173

1174 AI Pattern Alert Banner (conditional):

1175 - Shows in red/orange if any complaint category has 3+ this week

1176 - "🚨 4 WiFi complaints this week – possible infrastructure issue"

1177

1178 SCREEN 3: Complaints Management

1179 - Tabs: All | Pending | In Progress | Resolved

1180 - Complaint table columns:

1181 ID | Student Name | Category | Priority | Description (truncated) |

1182 Submitted | Status | Actions

1183 - Priority column colored: High=red, Medium=orange, Low=blue

1184 - Actions: "View Details" button on each row

1185 - Bulk action: "Mark Selected as In Progress"

1186

1187 SCREEN 4: Complaint Detail

1188 - Student info: name, room number, phone

1189 - Complaint: category badge, priority badge, full description, submitted time

1190 - AI Suggestion box: "💡 AI Suggested: [suggestion text]"

1191 - Status update dropdown: Pending → In Progress → Resolved

1192 - Warden notes text area

1193 - Save button

1194 - Activity log: shows status change history

1195

1196 SCREEN 5: Attendance Management

1197 - Date picker at top (default: today)

1198 - Student list with check-in status for selected date:

1199 Name | Room | Check-in Time | Check-out Time | Status

1200 - Status: Present (green), Absent (red), Not Yet (grey)

1201 - Manual mark button for each student (for demo – in real system it's biometric)

1202 - Export attendance button (fake for demo)

1203

1204 SCREEN 6: Student Roster

1205 - Table: Name | Room Number | University | Phone | Booking Status | Actions

1206 - Search bar to filter by name

1207 - "View Profile" action: shows student detail modal with their complaints and

1208 attendance history

1209

1210 SCREEN 7: Analytics

1211 - Date range selector

1212 - Charts (use Chart.js or Recharts):

1213 - Complaint volume over time (line chart)

1214 - Complaints by category (pie chart or bar chart)

1215 - Attendance rate over time (line chart)

1216 - Average complaint resolution time (gauge or number)

1217 - Data comes from GET /api/analytics/warden?hostel_id=

1218

1219 ----- E. ADMIN PORTAL (BASIC) -----

1220

1221 SCREEN 1: Admin Login and Dashboard

1222 - Stats: total hostels, total students, pending verifications, open disputes

1223

1224 SCREEN 2: Hostel Verification

1225 - List of hostels with their verification status

1226 - Columns: Name | Owner | Location | Submitted | Status | Actions

1227 - "View Details" shows hostel info and uploaded documents list

1228 - Approve / Reject buttons

1229 - Calls PATCH /api/hostels/{id}/verify

1230

1231 SCREEN 3: User Management (basic)

1232 - Table of all users with role, email, registration date, status

1233 - Deactivate account button

1234

1235 ----- F. DEPLOYMENT PLAN -----

1236

1237 For demo day, you need everything accessible from one network.

1238

1239 Option 1 (Simplest): All on localhost

1240 - Backend on localhost:8000

1241 - ML service on localhost:8001

1242 - Student app on localhost:3000

1243 - Warden dashboard on localhost:3001

1244 - Parent portal on localhost:3002

1245 - All team members on same WiFi network

1246 - Access from any device via your laptop's IP address:

1247 http://192.168.X.X:3000 (check your IP with ipconfig/ifconfig)

1248

1249 Option 2 (Better for demo): Render.com / Railway.app

1250 - Deploy backend to Render free tier

1251 - Deploy student app to Vercel free tier (just "vercel deploy")

1252 - All accessible from internet – no network dependency

1253 - Downside: free tier cold starts (first request is slow)

1254 - To fix cold start: set up an uptime monitor to ping the service

1255 (uptimerobot.com – free, pings every 5 minutes to keep it warm)

1256

1257 Recommendation: Use Option 2 if you have time. Deploy by May 7.

1258 Otherwise use Option 1 and ensure all devices are on same network on demo day.

1259

1260 One-command startup script (Option 1):

```
1261 Create start.sh:
1262     #!/bin/bash
1263     echo "Starting StayBuddy..."
1264     cd backend && python -m uvicorn app:app --port 8000 &
1265     cd ml_service && python -m uvicorn app:app --port 8001 &
1266     cd frontend/student && npm start -- --port 3000 &
1267     cd frontend/warden && npm start -- --port 3001 &
1268     cd frontend/parent && npm start -- --port 3002 &
1269     echo "All services started."
1270
1271 ----- G. DATA SEEDING SCRIPT -----
1272
1273 Your demo database needs to look REAL. Seed with:
1274
1275 Hostels (10-15 for demo):
1276     - Al-Noor Boys Hostel, F-8/1, Rs 8,000/month, 0.5km from FAST
1277     - Green Valley Residence, G-9, Rs 12,000/month, 1.2km from FAST
1278     - Sunrise Student Living, F-7, Rs 15,000/month, 0.8km from FAST
1279     - Islamabad Student Home, G-11, Rs 9,500/month, 2.1km from FAST
1280     - FAST Adjacent Hostel, F-8/4, Rs 18,000/month, 0.2km from FAST
1281     (Add 10 more with realistic Islamabad addresses)
1282
1283 Students (5 demo accounts):
1284     - ahmed@fast.edu.pk / password123 → has booking at Green Valley
1285     - sara@fast.edu.pk / password123 → has pending booking
1286     - usman@fast.edu.pk / password123 → no booking yet
1287     - ayesha@fast.edu.pk / password123 → has completed booking + reviews
1288     - bilal@fast.edu.pk / password123 → filed complaints
1289
1290 Wardens (2 demo accounts):
1291     - warden1@staybuddy.com / password123 → manages Green Valley
1292     - warden2@staybuddy.com / password123 → manages Al-Noor
1293
1294 Parents (2 demo accounts):
1295     - parent.ahmed@gmail.com / password123 → linked to ahmed's account
1296     - parent.sara@gmail.com / password123 → linked to sara's account
1297
1298 Admin:
1299     - admin@staybuddy.com / password123
1300
1301 Complaints (pre-seed some for demo):
1302     - Complaint from ahmed: WiFi not working, Medium, Pending → with AI suggestion
1303     - Complaint from ayesha: Room cleanliness, Low, Resolved
1304     - 3 WiFi complaints this week → triggers pattern alert on warden dashboard
1305
```

1306 Attendance (pre-seed):
1307 - 7 days of attendance data for ahmed and sara
1308 - All check-in times between 9 PM - 11 PM (realistic for students)
1309
1310
1311 =====
1312 9. INTEGRATION CHECKPOINTS
1313 =====
1314
1315 CHECKPOINT 1 (April 26 – Saturday):
1316 ✓ Student can log in
1317 ✓ Student sees recommendations on home screen (from real ML model)
1318 ✓ Student can click a hostel and see real detail from DB
1319 ✓ Chatbot API responds to at least 3 intent types
1320 ✓ Warden can log in and see complaints list
1321
1322 CHECKPOINT 2 (May 3 – Saturday):
1323 ✓ Full student journey works: login → recommend → detail → book → confirm
1324 ✓ Chatbot handles all 7 intents correctly
1325 ✓ Parent portal shows real child data
1326 ✓ Warden complaint workflow works: view → update status → student sees update
1327 ✓ Emergency SOS: student triggers → warden and parent see notification
1328 ✓ Admin can log in and see verification dashboard
1329 ✓ All API responses using real DB data (zero mocks remaining)
1330
1331 CHECKPOINT 3 (May 7 – Wednesday):
1332 ✓ Presentation slides complete
1333 ✓ Demo script rehearsed and timed
1334 ✓ All known bugs fixed
1335 ✓ App deployed or deployment-ready
1336 ✓ Demo database populated with clean data
1337 ✓ Backup video recorded
1338 ✓ ML metrics printed and slide-ready
1339
1340
1341 =====
1342 10. DATABASE SCHEMA REFERENCE
1343 =====
1344
1345 Table: users
1346 id SERIAL PRIMARY KEY
1347 name VARCHAR(100) NOT NULL
1348 email VARCHAR(150) UNIQUE NOT NULL
1349 password_hash VARCHAR(255) NOT NULL
1350 role ENUM('student', 'parent', 'warden', 'admin') NOT NULL

```
1351     phone            VARCHAR(20)
1352     profile_photo    VARCHAR(500)    -- URL or path
1353     is_active         BOOLEAN DEFAULT TRUE
1354     created_at        TIMESTAMP DEFAULT NOW()
1355
1356 Table: students (extends users)
1357     user_id           INTEGER REFERENCES users(id) ON DELETE CASCADE PRIMARY KEY
1358     university        VARCHAR(100)
1359     budget_min         INTEGER
1360     budget_max         INTEGER
1361     commute_max_km     DECIMAL(4,2)
1362     preference_wifi     BOOLEAN DEFAULT FALSE
1363     preference_ac       BOOLEAN DEFAULT FALSE
1364     preference_meals    BOOLEAN DEFAULT FALSE
1365     preference_gym      BOOLEAN DEFAULT FALSE
1366     preference_laundry  BOOLEAN DEFAULT FALSE
1367     preference_study    BOOLEAN DEFAULT FALSE
1368     study_vs_social    DECIMAL(3,2)  -- 0 = social, 1 = study
1369
1370 Table: parent_student_links
1371     id                 SERIAL PRIMARY KEY
1372     parent_id          INTEGER REFERENCES users(id)
1373     student_id         INTEGER REFERENCES users(id)
1374
1375 Table: hostels
1376     id                 SERIAL PRIMARY KEY
1377     name                VARCHAR(150) NOT NULL
1378     address             VARCHAR(300) NOT NULL
1379     latitude            DECIMAL(9,6)
1380     longitude           DECIMAL(9,6)
1381     distance_from_campus_km DECIMAL(5,2)
1382     description         TEXT
1383     warden_id          INTEGER REFERENCES users(id)
1384     total_capacity      INTEGER
1385     current_occupancy  INTEGER DEFAULT 0
1386     overall_rating     DECIMAL(3,2)
1387     cleanliness_rating DECIMAL(3,2)
1388     facilities_rating  DECIMAL(3,2)
1389     management_rating  DECIMAL(3,2)
1390     location_rating    DECIMAL(3,2)
1391     verification_status ENUM('pending', 'approved', 'rejected') DEFAULT 'pending'
1392     created_at         TIMESTAMP DEFAULT NOW()
1393
1394 Table: hostel_amenities
1395     id                 SERIAL PRIMARY KEY
```

```

1396     hostel_id      INTEGER REFERENCES hostels(id) ON DELETE CASCADE
1397     amenity_name    VARCHAR(100)    -- 'WiFi', 'AC', 'Meals', 'Gym', 'Laundry', etc.
1398     is_available    BOOLEAN DEFAULT TRUE
1399
1400 Table: room_types
1401     id              SERIAL PRIMARY KEY
1402     hostel_id       INTEGER REFERENCES hostels(id) ON DELETE CASCADE
1403     type_name       VARCHAR(50)    -- 'Single', 'Double', 'Triple', 'Dormitory'
1404     price_per_month INTEGER
1405     total_beds      INTEGER
1406     available_beds  INTEGER
1407     description     TEXT
1408
1409 Table: bookings
1410     id              SERIAL PRIMARY KEY
1411     student_id      INTEGER REFERENCES users(id)
1412     hostel_id       INTEGER REFERENCES hostels(id)
1413     room_type_id    INTEGER REFERENCES room_types(id)
1414     check_in_date   DATE
1415     duration_months INTEGER
1416     total_amount    INTEGER
1417     status          ENUM('pending', 'confirmed', 'cancelled', 'completed')
1418     special_requirements TEXT
1419     created_at      TIMESTAMP DEFAULT NOW()
1420
1421 Table: complaints
1422     id              SERIAL PRIMARY KEY
1423     student_id      INTEGER REFERENCES users(id)
1424     hostel_id       INTEGER REFERENCES hostels(id)
1425     category        ENUM('Maintenance', 'Food', 'Cleanliness', 'Security',
1426                          'Internet', 'Management', 'Other')
1427     severity        ENUM('low', 'medium', 'high')
1428     description     TEXT NOT NULL
1429     ai_suggestion   TEXT            -- populated by NLP model
1430     status          ENUM('pending', 'in_progress', 'resolved') DEFAULT 'pending'
1431     assigned_to     INTEGER REFERENCES users(id) -- warden assigned
1432     warden_notes    TEXT
1433     created_at      TIMESTAMP DEFAULT NOW()
1434     updated_at      TIMESTAMP DEFAULT NOW()
1435
1436 Table: complaint_history
1437     id              SERIAL PRIMARY KEY
1438     complaint_id    INTEGER REFERENCES complaints(id)
1439     changed_by      INTEGER REFERENCES users(id)
1440     old_status      VARCHAR(50)

```



```

1441     new_status      VARCHAR(50)
1442     note            TEXT
1443     changed_at      TIMESTAMP DEFAULT NOW()
1444
1445 Table: attendance
1446     id              SERIAL PRIMARY KEY
1447     student_id      INTEGER REFERENCES users(id)
1448     hostel_id       INTEGER REFERENCES hostels(id)
1449     event_type      ENUM('check_in', 'check_out')
1450     event_time      TIMESTAMP DEFAULT NOW()
1451     logged_by       VARCHAR(50) DEFAULT 'manual' -- 'biometric', 'manual', 'auto'
1452
1453 Table: reviews
1454     id              SERIAL PRIMARY KEY
1455     student_id      INTEGER REFERENCES users(id)
1456     hostel_id       INTEGER REFERENCES hostels(id)
1457     booking_id      INTEGER REFERENCES bookings(id)
1458     overall_rating  INTEGER CHECK (overall_rating BETWEEN 1 AND 5)
1459     cleanliness     INTEGER CHECK (cleanliness BETWEEN 1 AND 5)
1460     facilities      INTEGER CHECK (facilities BETWEEN 1 AND 5)
1461     management      INTEGER CHECK (management BETWEEN 1 AND 5)
1462     location        INTEGER CHECK (location BETWEEN 1 AND 5)
1463     review_text     TEXT
1464     created_at      TIMESTAMP DEFAULT NOW()
1465
1466 Table: favorites
1467     id              SERIAL PRIMARY KEY
1468     student_id      INTEGER REFERENCES users(id)
1469     hostel_id       INTEGER REFERENCES hostels(id)
1470     created_at      TIMESTAMP DEFAULT NOW()
1471     UNIQUE(student_id, hostel_id)
1472
1473 Table: student_interactions
1474     id              SERIAL PRIMARY KEY
1475     student_id      INTEGER REFERENCES users(id)
1476     hostel_id       INTEGER REFERENCES hostels(id)
1477     action_type     ENUM('view', 'search', 'favorite', 'booking_attempt',
1478                          'booking_complete', 'review')
1479     created_at      TIMESTAMP DEFAULT NOW()
1480
1481 Table: notifications
1482     id              SERIAL PRIMARY KEY
1483     user_id         INTEGER REFERENCES users(id)
1484     type            VARCHAR(50) -- 'attendance', 'complaint_update', 'emergency',
1485                          -- 'booking_confirmed', 'fee_reminder'

```

```
1486 title          VARCHAR(200)
1487 message         TEXT
1488 related_id       INTEGER      -- ID of complaint, booking, etc.
1489 is_read          BOOLEAN DEFAULT FALSE
1490 created_at       TIMESTAMP DEFAULT NOW()
```

1492 Table: emergency_alerts

```
1493 id              SERIAL PRIMARY KEY
1494 student_id       INTEGER REFERENCES users(id)
1495 hostel_id        INTEGER REFERENCES hostels(id)
1496 alert_type       VARCHAR(100)
1497 latitude         DECIMAL(9,6)
1498 longitude        DECIMAL(9,6)
1499 status           ENUM('active', 'resolved') DEFAULT 'active'
1500 resolved_by      INTEGER REFERENCES users(id)
1501 created_at       TIMESTAMP DEFAULT NOW()
```

1503 INDEXES TO CREATE:

```
1504 CREATE INDEX idx_complaints_hostel ON complaints(hostel_id);
1505 CREATE INDEX idx_complaints_student ON complaints(student_id);
1506 CREATE INDEX idx_attendance_student ON attendance(student_id);
1507 CREATE INDEX idx_attendance_hostel ON attendance(hostel_id);
1508 CREATE INDEX idx_bookings_student ON bookings(student_id);
1509 CREATE INDEX idx_bookings_hostel ON bookings(hostel_id);
1510 CREATE INDEX idx_interactions_student ON student_interactions(student_id);
1511 CREATE INDEX idx_notifications_user ON notifications(user_id);
```

```
1514 =====
1515 11. API CONTRACT REFERENCE
1516 =====
```

1518 (The exact request/response shapes all 3 of you must agree on and not deviate from)

1520 POST /auth/login

```
1521 Request: { "email": "string", "password": "string" }
1522 Response: { "token": "JWT_STRING", "user": { "id", "name", "email", "role" } }
1523 Error 401: { "error": "Invalid credentials" }
```

1525 POST /api/recommendations

```
1526 Request: { "student_id": 123, "top_k": 10 }
1527 Response: {
1528     "recommendations": [{
1529         "hostel_id": 45,
1530         "hostel_name": "string",
```

```
1531     "match_score": 0.87,
1532     "top_matching_factors": ["string"],
1533     "price_per_month": 12000,
1534     "distance_km": 0.8,
1535     "rating": 4.2,
1536     "available_beds": 3
1537   }]
1538 }
1539
1540 GET /api/hostels
1541   Query params: price_min, price_max, distance_max, amenities, room_type, sort_by
1542   Response: {
1543     "hostels": [{ all hostel fields + amenities list + room_types list }],
1544     "total": 50
1545   }
1546
1547 GET /api/hostels/{id}
1548   Response: {
1549     "id", "name", "address", "latitude", "longitude",
1550     "distance_from_campus_km", "description", "warden_id",
1551     "overall_rating", "cleanliness_rating", "facilities_rating",
1552     "management_rating", "location_rating", "verification_status",
1553     "amenities": [{ "amenity_name", "is_available" }],
1554     "room_types": [{ "type_name", "price_per_month", "available_beds" }],
1555     "warden": { "name", "phone" }
1556   }
1557
1558 POST /api/bookings
1559   Request: {
1560     "student_id": 123,
1561     "hostel_id": 45,
1562     "room_type_id": 3,
1563     "check_in_date": "2026-05-01",
1564     "duration_months": 6,
1565     "special_requirements": "string"
1566   }
1567   Response: {
1568     "booking_id": 789,
1569     "status": "confirmed",
1570     "total_amount": 72000,
1571     "hostel_name": "Green Valley",
1572     "room_type": "Single"
1573   }
1574
1575 POST /api/complaints
```

```
1576 Request: {
1577     "student_id": 123,
1578     "hostel_id": 45,
1579     "category": "Internet",
1580     "severity": "medium",
1581     "description": "WiFi has been down for 2 days"
1582 }
1583 Response: {
1584     "complaint_id": 55,
1585     "status": "pending",
1586     "ai_suggestion": "Reset the router or contact ISP",
1587     "message": "Complaint filed successfully. Your warden has been notified."
1588 }
1589
1590 POST /api/chatbot
1591 Request: {
1592     "message": "string",
1593     "student_id": 123,
1594     "conversation_history": [
1595         {"role": "user", "message": "string"},
1596         {"role": "bot", "message": "string"}
1597     ]
1598 }
1599 Response: {
1600     "response": "string",
1601     "intent": "string",
1602     "entities": {},
1603     "quick_replies": ["string"]
1604 }
1605
1606 POST /api/emergency
1607 Request: { "student_id": 123, "alert_type": "SOS", "latitude": 33.6, "longitude": 73.0 }
1608 Response: { "alert_id": 1, "status": "active",
1609     "message": "Emergency alert sent to warden and parents" }
1610
1611 GET /api/notifications?user_id=123
1612 Response: {
1613     "notifications": [{
1614         "id", "type", "title", "message", "related_id", "is_read", "created_at"
1615     }],
1616     "unread_count": 3
1617 }
1618
1619 GET /api/analytics/warden?hostel_id=5
1620 Response: {
```

```
1621     "total_students": 45,
1622     "total_complaints_this_month": 12,
1623     "resolved_complaints": 8,
1624     "resolution_rate": 0.67,
1625     "avg_resolution_hours": 18,
1626     "attendance_rate_this_week": 0.89,
1627     "complaint_categories": [
1628         { "category": "Internet", "count": 5 },
1629         { "category": "Cleanliness", "count": 3 },
1630         { "category": "Maintenance", "count": 4 }
1631     ]
1632 }
1633
1634
1635 =====
1636 12. THE DEMO FLOW (TARGET END STATE)
1637 =====
1638
1639 This is the story you tell on May 10th. Every screen you build should make
1640 this flow possible. Practice this exact sequence until it's smooth.
1641
1642 ESTIMATED TIME: 8-10 minutes
1643
1644 --- SETUP BEFORE DEMO ---
1645     - All devices charged and connected to internet
1646     - Backend running (or deployed)
1647     - Demo database seeded with clean data
1648     - All three portals open and ready on their respective devices:
1649         Device 1 (Eraj presents): Student app – logged in as ahmed@fast.edu.pk
1650         Device 2 (Samiya presents): Parent portal – logged in as parent.ahmed@gmail.com
1651         Device 3 (Zarnab presents): Warden dashboard – logged in as warden1@staybuddy.com
1652
1653 --- THE STORY ---
1654
1655 [Eraj – Student App]
1656 "Ahmed is a second-year CS student at FAST who just moved to Islamabad
1657 and needs to find a hostel near campus within his budget of 15,000 rupees."
1658
1659 1. Show student login → log in as Ahmed
1660 2. Show home screen: "StayBuddy immediately shows Ahmed personalized
1661     hostel recommendations based on his preferences – budget, distance,
1662     and desired amenities like WiFi and a study room."
1663 3. Show recommendation cards with match scores:
1664     "Notice the match score – 91% means this hostel closely matches
1665     Ahmed's stated preferences. Let me click on Green Valley."
```

1666 4. Show hostel detail: amenities tab, rooms tab, rating breakdown
1667 "You can see all amenities, pricing by room type, and student reviews."
1668 5. Click Book Now → room selection → confirmation
1669 "Ahmed books a single room for 6 months. Booking confirmed."
1670
1671 [Samiya – Chatbot, then Parent Portal]
1672 6. Navigate to Chat screen on student app:
1673 "Ahmed has a question about the hostel."
1674 Type: "Does Green Valley have a study room?"
1675 Show bot responding correctly with entity extraction working
1676 Type: "How far is it from campus?"
1677 Show context-aware response using "it" to refer to Green Valley
1678
1679 7. Ahmed types: "The WiFi has been down for 2 days"
1680 Bot detects complaint intent and files complaint automatically.
1681 "The chatbot identified this as a complaint, categorized it as Internet,
1682 and filed it automatically. Ahmed gets a confirmation with complaint ID."
1683
1684 8. Switch to Parent Portal device:
1685 "Meanwhile, Ahmed's parent receives a notification that his booking
1686 was confirmed, and can see all his details."
1687 Show parent home: last check-in, current hostel, attendance stats
1688 "The parent can see Ahmed's attendance history, live hostel location,
1689 and contact the warden directly."
1690
1691 [Zarnab – Warden Dashboard]
1692 9. Switch to Warden Dashboard:
1693 "The warden immediately sees the new complaint filed through the chatbot."
1694 Show complaints list – new complaint is at the top
1695 Click complaint detail:
1696 "The AI has already suggested a resolution action based on the
1697 complaint category. The warden can update the status with one click."
1698 Update status to "In Progress"
1699
1700 10. Show the pattern alert banner if 3+ WiFi complaints:
1701 "StayBuddy has detected a pattern – 4 WiFi complaints this week.
1702 This proactive alert helps wardens identify systemic issues early."
1703
1704 11. Show analytics screen: charts for complaint trends and attendance rate.
1705 "The warden has a complete analytics view of their hostel's performance."
1706
1707 [Eraj – Back to Student App]
1708 12. Show SOS button demonstration:
1709 "In an emergency, students tap the SOS button."
1710 Tap it – show success message.

1711 Switch to parent portal – show emergency alert notification.
1712 Switch to warden dashboard – show emergency alert.
1713 "All stakeholders are instantly notified and can coordinate a response."
1714
1715 [CLOSING]
1716 "StayBuddy connects students, parents, and wardens in one intelligent ecosystem.
1717 The AI recommendation engine, NLP chatbot, and automated complaint management
1718 work together to solve real problems for Pakistan's student housing sector."
1719
1720 --- POST DEMO: SHOW METRICS ---
1721 Eraj: Recommendation engine metrics slide
1722 - Precision@5: X%
1723 - Mean Average Precision: X%
1724 - RMSE: X
1725 - Coverage: X% of hostels recommended
1726
1727 Samiya: Chatbot metrics slide
1728 - Intent Classification Accuracy: X%
1729 - F1 Score per intent (table)
1730 - Entity Extraction Accuracy: X%
1731 - Confusion matrix (screenshot)
1732
1733 Zarnab: Infrastructure metrics slide
1734 - Average API Response Time: Xms
1735 - Slowest endpoint: Xms
1736 - Database record counts: X hostels, X students, X complaints
1737 - Zero data integrity violations
1738
1739
1740 =====
1741 13. TESTING CHECKLIST
1742 =====
1743
1744 Run through this checklist on May 3rd and again on May 7th.
1745 Mark each item PASS / FAIL / N/A
1746
1747 AUTHENTICATION:
1748 [] Student registration creates user and returns token
1749 [] Student login returns correct token
1750 [] Invalid password returns 401
1751 [] JWT token correctly restricts warden endpoints from student
1752 [] Parent login shows parent-specific data only
1753
1754 RECOMMENDATION ENGINE:
1755 [] POST /api/recommendations returns 10 ranked hostels

1756 [] Match scores are between 0 and 1
1757 [] Top matching factors are included and accurate
1758 [] Different student IDs return different rankings
1759 [] Response time under 2 seconds
1760
1761 CHATBOT:
1762 [] Hostel search intent recognized correctly
1763 [] Amenity inquiry extracts hostel name and amenity correctly
1764 [] Pricing intent returns real price from DB
1765 [] Location intent extracts location reference
1766 [] Complaint intent creates complaint in DB
1767 [] Unknown input returns graceful fallback message
1768 [] Context maintained: "it" refers to previously mentioned hostel
1769 [] Response time under 3 seconds
1770
1771 STUDENT APP:
1772 [] Login → home screen loads with recommendations
1773 [] Search with price filter returns correctly filtered results
1774 [] Hostel detail shows real data (amenities, rooms, reviews)
1775 [] Booking flow completes and creates booking in DB
1776 [] Chat screen sends message and receives response
1777 [] Complaint submission creates complaint in DB
1778 [] SOS button creates emergency alert in DB
1779 [] Favorites add/remove works correctly
1780 [] All screens load within 3 seconds
1781
1782 WARDEN DASHBOARD:
1783 [] Warden sees only their hostel's complaints
1784 [] Status update on complaint reflects immediately
1785 [] AI suggestion appears on complaint detail
1786 [] Pattern alert banner shows when 3+ complaints in same category
1787 [] Attendance marking creates attendance record in DB
1788 [] Analytics charts load with real data
1789
1790 PARENT PORTAL:
1791 [] Parent sees only their linked child's data
1792 [] Attendance history shows correct calendar view
1793 [] Notifications list shows booking confirmed, complaint updates
1794 [] Emergency alert appears when SOS is triggered
1795
1796 DATABASE:
1797 [] No orphaned records (booking with invalid student_id, etc.)
1798 [] All foreign key constraints hold
1799 [] Attendance records have valid timestamps
1800 [] Complaint categories are from the valid enum list

1801 [] Review ratings are between 1 and 5

1802

1803 API GENERAL:

1804 [] All endpoints return proper HTTP status codes

1805 [] Error responses have { "error": "message" } format

1806 [] CORS configured for all frontend origins

1807 [] No endpoints leaking password hashes

1808

1809

1810 =====

1811 14. PRESENTATION STRUCTURE

1812 =====

1813

1814 SLIDE DECK: 15-20 slides, target 5-6 minutes of presentation

1815

1816 Slide 1 – Title

1817 StayBuddy: Your Intelligent Hostel Companion

1818 Team names, roll numbers, supervisor

1819

1820 Slide 2 – The Problem (30 seconds)

1821 5 problems from your proposal, shown cleanly

1822 "X million students in Pakistan need safe, affordable accommodation"

1823

1824 Slide 3 – Our Solution (30 seconds)

1825 One-liner per feature: Recommendation Engine, AI Chatbot,

1826 Complaint Management, Parent Monitoring

1827 Show your app logo/tagline

1828

1829 Slide 4 – System Architecture (1 minute)

1830 Clean diagram showing:

1831 Client Layer → API Gateway → Backend Services → AI/ML Layer → Data Layer

1832 Show how all 3 components connect

1833

1834 Slide 5 – Recommendation Engine (Eraj – 1 minute)

1835 What it does, how it works (content + collaborative + hybrid)

1836 METRICS BOX: Precision@5, MAP, RMSE, Coverage

1837

1838 Slide 6 – Recommendation Demo Screenshot

1839 Screenshot of recommendation results screen with match scores visible

1840

1841 Slide 7 – AI Chatbot (Samiya – 1 minute)

1842 What it does, how it works (intent classification + entity extraction)

1843 METRICS BOX: Accuracy, F1 scores table, confusion matrix thumbnail

1844

1845 Slide 8 – Chatbot Demo Screenshot

1846 Screenshot of chat conversation showing intent working

1847

1848 Slide 9 – Complaint Management AI

1849 How NLP categorizes and suggests resolutions

1850 Show the warden complaint detail with AI suggestion visible

1851

1852 Slide 10 – Data Infrastructure (Zarnab – 1 minute)

1853 Database schema overview (simplified ERD)

1854 API endpoints count, response times

1855 Tech stack used

1856

1857 Slide 11 – Parent Portal Demo

1858 Screenshot showing parent dashboard with child status

1859

1860 Slide 12 – Warden Dashboard Demo

1861 Screenshot showing complaint management and analytics

1862

1863 Slide 13 – Tech Stack Summary

1864 Clean grid: Frontend | Backend | AI/ML | Database | Deployment

1865

1866 Slide 14 – Challenges & Solutions

1867 3 real challenges you faced and how you solved them

1868 This shows evaluators you actually built it

1869

1870 Slide 15 – Future Work

1871 Voice search, biometric integration, payment gateway,

1872 geofencing, social recommendations

1873

1874 Slide 16 – Conclusion

1875 "StayBuddy connects students, parents, and wardens in one intelligent ecosystem"

1876 GitHub repo QR code or link

1877

1878 COMMON EVALUATOR QUESTIONS & ANSWERS:

1879

1880 Q: Why did you use a hybrid recommendation model instead of just one approach?

1881 A: Content-based filtering alone can't discover hidden patterns – it only matches

1882 stated preferences. Collaborative filtering alone struggles with new users who

1883 have no interaction history (cold start problem). The hybrid model addresses

1884 both limitations. Content-based serves new users, collaborative improves over time.

1885

1886 Q: How does your chatbot handle Urdu queries?

1887 A: For the current implementation, we support English. Our training data covers

1888 formal and informal English phrasing. Multilingual support using mBERT or

1889 XLM-R is planned for Phase 2.

1890

1891 Q: How do you handle the cold start problem for new students?

1892 A: For new students with no interaction history, we rely purely on content-based

1893 filtering using their stated preferences from the registration form.

1894 As they interact with the system, collaborative signals strengthen.

1895

1896 Q: What if the recommendation model data is synthetic – how will it perform on real data?

1897 A: Our synthetic dataset was designed to reflect realistic student decision patterns

1898 including budget constraints, distance preferences, and amenity priorities.

1899 The model architecture itself is not tied to synthetic data – with real interaction

1900 data it will only improve. The current metrics establish a performance baseline.

1901

1902 Q: Is the system secure?

1903 A: We use JWT authentication, bcrypt password hashing, role-based access control,

1904 parameterized SQL queries to prevent injection, and HTTPS for all API calls.

1905

1906 Q: How does the AI complaint categorization work?

1907 A: Samiya's NLP model classifies complaint text into categories. The model was

1908 trained on labeled complaint examples and achieves X% accuracy on test data.

1909 The category determines which resolution template is suggested to the warden.

1910

1911 Q: What makes this different from existing hostel listing websites in Pakistan?

1912 A: Existing platforms are static listings with no intelligence. StayBuddy adds:

1913 (1) personalized AI recommendations that learn from behavior,

1914 (2) intelligent chatbot that understands natural language,

1915 (3) real-time parent monitoring and alerts,

1916 (4) AI-powered complaint management connecting all stakeholders.

1917 No existing platform in Pakistan integrates all four.

1918

1919

1920 =====

1921 15. EMERGENCY BACKUP PLANS

1922 =====

1923

1924 SCENARIO: Recommendation API is down during demo

1925 Backup: Have a JSON file of pre-computed recommendations for 3 demo student IDs.

1926 Your frontend should check: if API fails → load from fallback JSON.

1927 The evaluator will never know the difference.

1928

1929 SCENARIO: Chatbot gives wrong intent during demo

1930 Backup: Practice with 10 specific test queries that you KNOW work.

1931 Only use those exact queries during the demo. Don't improvise.

1932 Have the phrases written on a sticky note.

1933

1934 SCENARIO: Internet is down on demo day

1935 Backup: Everything running on localhost.

1936 Have the backend and all frontends ready to run locally.

1937 Keep the demo database seeded and ready on your local machine.

1938

1939 SCENARIO: Database connection fails

1940 Backup: Have SQLite as a fallback (same schema, file-based, no server needed).

1941 Alternatively: keep a full exported SQL dump and be ready to restore in 2 minutes.

1942

1943 SCENARIO: Frontend crashes mid-demo

1944 Backup: Have a screen recording video of the full demo flow working perfectly.

1945 If anything crashes: "Let me show you the pre-recorded demo while we restart."

1946 Record this video by May 7th.

1947

1948 SCENARIO: One team member is sick on demo day

1949 Backup: Every team member should know the basics of all three components.

1950 Eraj should be able to explain chatbot briefly if Samiya is absent.

1951 Spend 30 minutes briefing each other on April 28th.

1952

1953 GENERAL TIPS FOR DEMO DAY:

1954 - Have everything pre-loaded – no typing URLs during demo

1955 - Use incognito/private windows to avoid browser cache issues

1956 - Disable browser notifications before demo starts

1957 - Charge all devices the night before

1958 - Have the video backup on your phone, not just your laptop

1959 - If something breaks: stay calm, say "Let me switch to the backup"

1960 - Evaluators expect things to not work perfectly – confidence matters more

1961

1962

1963 =====

1964 16. DAILY STANDUP TEMPLATE

1965 =====

1966

1967 Every day at 9:00 PM (or whatever time works), meet for 15 minutes:

1968

1969 ERAJ:

1970 Yesterday I completed: [list specific screens/features]

1971 Today I will complete: [specific screens/features]

1972 Blockers: [any APIs from Zarnab I'm waiting on? Any design questions?]

1973

1974 SAMIYA:

1975 Yesterday I completed: [chatbot intents done, screens built]

1976 Today I will complete: [specific tasks]

1977 Blockers: [any DB endpoints from Zarnab? Any chatbot performance issues?]

1978

1979 ZARNAB:

1980 Yesterday I completed: [endpoints deployed, dashboard screens built]

1981 Today I will complete: [specific endpoints/screens]
1982 Blockers: [any unclear data formats from Eraj or Samiya?]
1983
1984 Integration issue to resolve today: [if any]
1985 Tomorrow's priority: [agree on top priority for next day]
1986
1987 RULE: If you're blocked, say so immediately – don't wait for standup.
1988 Use your group chat for urgent issues.
1989
1990
1991 =====
1992 END OF STAYBUDDY IMPLEMENTATION PLAN
1993 =====
1994
1995 Team: Eraj Zaman (221-1296) | Samiya Saleem (221-1065) | Zarnab (221-1005)
1996 Supervisor: Dr. Ahkter Jamil
1997 Project: StayBuddy – Your Intelligent Hostel Companion
1998 Evaluation: May 10, 2026
1999
2000 Good luck. Build something you're proud of.
2001
2002 =====
2003